

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №5
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Определение характеристик графов»

Выполнили студенты группы 25BBB2

Лампетов А.С.

Ковтун А.А.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В.

Пенза 2026

Тема: Определение характеристик графов

Цель работы:

изучение основных понятий теории графов и способов их представления в памяти компьютера; реализация на языке Go генерации матрицы смежности и построения матрицы инцидентности неориентированного графа; определение размера графа, степеней вершин, а также изолированных, концевых и доминирующих вершин.

Лабораторное задание:

1. Сгенерировать (используя генератор случайных чисел) матрицу смежности для неориентированного графа G и вывести её на экран.
2. Определить размер графа G , используя матрицу смежности.
3. Найти изолированные, концевые и доминирующие вершины.
4. * Построить для графа G матрицу инцидентности.
5. * Определить размер графа G , используя матрицу инцидентности.
6. * Найти изолированные, концевые и доминирующие вершины по матрице инцидентности.

Ход работы:

Граф G является неориентированным, без петель и кратных рёбер. Он содержит $N = 6$ вершин, а вероятность наличия ребра между любой парой вершин задана константой $P = 0,4$. Вершины обозначаются $v_1, \dots, v_6$. Граф хранится в виде матрицы смежности — двумерного среза целых чисел размера $N \times N$, где элемент $[i][j]$ равен 1, если вершины v_i и v_j соединены ребром, и 0 в противном случае.

Функция `generateAdjacencyMatrix` создаёт нулевую матрицу и обходит только элементы выше главной диагонали ($j > i$). Для каждой пары вершин генератор случайных чисел выдаёт значение от 0 до 1; если оно не превышает P , ребро добавляется: единица записывается одновременно в позиции $[i][j]$ и $[j][i]$. Благодаря этому матрица получается симметричной (граф неориентированный), а главная диагональ остаётся нулевой (петель нет).

Генератор инициализируется текущим временем

`rand.NewSource(time.Now().UnixNano())`, поэтому при каждом запуске строится новый граф.

```
1 func generateAdjacencyMatrix(n int, p float64, rng *rand.Rand) [][]int {
2     matrix := make([][]int, n)
3     for i := range matrix {
4         matrix[i] = make([]int, n)
5     }
6     for i := 0; i < n; i++ {
7         for j := i + 1; j < n; j++ {
8             if rng.Float64() <= p {
9                 matrix[i][j] = 1
10                matrix[j][i] = 1
11            }
12        }
13    }
14    return matrix
15 }
```

Рисунок 1 — Функция генерации матрицы смежности

Вывод матриц на экран выполняет функция `printMatrix`, которая принимает матрицу, подписи строк и столбцов и заголовок. Для матрицы смежности и строки, и столбцы подписываются именами вершин, а для матрицы инцидентности столбцы подписываются именами рёбер $e_1, \dots, e_m$.

Размер графа, то есть количество его рёбер $|E(G)|$, по матрице смежности определяет функция `sizeGraph`. Она суммирует все элементы матрицы. Так как каждое ребро (v_i, v_j) записано в матрице дважды — в позициях $[i][j]$ и $[j][i]$, — сумма равна $2|E|$, и в `main` она делится на 2. Степень вершины $\deg(v_i)$ равна числу рёбер, инцидентных этой вершине, то есть сумме элементов i -й строки; её вычисляет функция `degreesFromAdjacency`.

```

1 func sizeGraph(adj [][]int) int {
2     sum := 0
3     for _, row := range adj {
4         for _, v := range row {
5             sum += v
6         }
7     }
8     return sum
9 }
10
11 func degreesFromAdjacency(matrix [][]int) []int {
12     n := len(matrix)
13     deg := make([]int, n)
14     for i := 0; i < n; i++ {
15         for j := 0; j < n; j++ {
16             deg[i] += matrix[i][j]
17         }
18     }
19     return deg
20 }

```

Рисунок 2 — Функции вычисления размера графа и степеней вершин

Классификация вершин выполняется функцией `analyzeVertices`, которая принимает срез степеней и число вершин `n` и возвращает структуру `VertexClassification` с тремя списками номеров вершин. Вершина считается изолированной, если её степень равна 0, концевой — если степень равна 1, и доминирующей — если степень равна $n - 1$, то есть вершина смежна со всеми остальными. Номера вершин сохраняются начиная с единицы ($i+1$), чтобы они соответствовали обозначениям $v_1, \dots, v_6$. Если подходящих вершин нет, функция `printIntSlice` выводит слово «нет».

```

1 type VertexClassification struct {
2     Isolated []int
3     Terminal []int
4     Dominating []int
5 }
6
7 func analyzeVertices(deg []int, n int) VertexClassification {
8     var res VertexClassification
9     for i, d := range deg {
10         switch {
11             case d == 0:
12                 res.Isolated = append(res.Isolated, i+1)
13             case d == 1:
14                 res.Terminal = append(res.Terminal, i+1)
15         }
16         if d == n-1 {
17             res.Dominating = append(res.Dominating, i+1)
18         }
19     }
20     return res
21 }

```

Рисунок 3 — Структура результата и функция классификации вершин

Для выполнения задания 2 реализована функция `buildIncidenceMatrix`. Сначала по матрице смежности формируется список рёбер: обход элементов выше главной диагонали гарантирует, что каждое ребро будет учтено ровно один раз, а рёбра получают номера $e_1, \dots, e_m$ в порядке обнаружения. Затем создаётся матрица размера $n \times m$, где строки соответствуют вершинам, а столбцы — рёбрам. Для ребра $e_k = (v_u, v_v)$ единицы записываются в строки u и v столбца k . Таким образом, в каждом столбце матрицы инцидентности ровно две единицы.

```

1 func buildIncidenceMatrix(adj [][]int) ([][]int, []Edge) {
2     n := len(adj)
3     var edges []Edge
4     for i := 0; i < n; i++ {
5         for j := i + 1; j < n; j++ {
6             if adj[i][j] == 1 {
7                 edges = append(edges, Edge{i, j})
8             }
9         }
10    }
11
12    m := len(edges)
13    inc := make([][]int, n)
14    for i := range inc {
15        inc[i] = make([]int, m)
16    }
17    for k, e := range edges {
18        inc[e.U][k] = 1
19        inc[e.V][k] = 1
20    }
21    return inc, edges
22 }

```

Рисунок 4 — Функция построения матрицы инцидентности

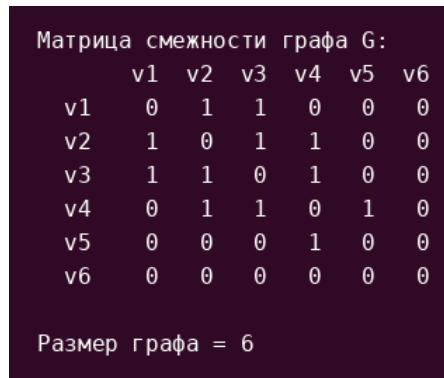
Размер графа по матрице инцидентности равен числу её столбцов, так как каждый столбец соответствует одному ребру. Степень вершины равна сумме элементов соответствующей строки, поскольку единицы в строке стоят на тех рёбрах, которым вершина инцидентна. Для классификации вершин по матрице инцидентности используется та же функция `analyzeVertices`, что и в задании 1, что позволяет сравнить результаты двух представлений графа.

Оценка сложности. Построение матрицы смежности и вычисление степеней по ней требуют $O(n^2)$ операций и $O(n^2)$ памяти. Матрица инцидентности занимает $O(n \cdot m)$ памяти, где m — число рёбер; для плотных

графов это больше, чем у матрицы смежности, а для разреженных — сопоставимо или меньше, если не хранить нулевые столбцы.

Результаты работы программы:

Ниже приведён результат работы программы для одного из запусков. Сгенерированный граф содержит 6 вершин. Заданию 1 соответствует вывод матрицы смежности и размера графа.

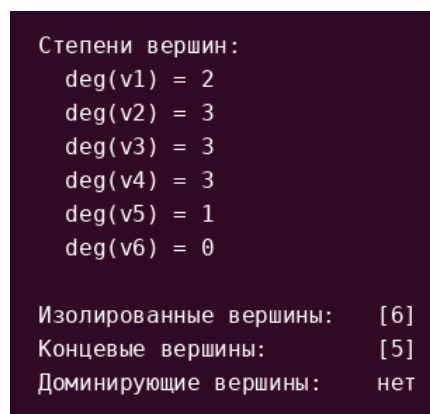


```
Матрица смежности графа G:
      v1 v2 v3 v4 v5 v6
v1    0  1  1  0  0  0
v2    1  0  1  1  0  0
v3    1  1  0  1  0  0
v4    0  1  1  0  1  0
v5    0  0  0  1  0  0
v6    0  0  0  0  0  0

Размер графа = 6
```

Рисунок 5 — Матрица смежности и размер графа (задание 1)

По матрице смежности вычислены степени вершин и найдены особые вершины. В данном графе вершина v_6 не имеет ни одного ребра и является изолированной, вершина v_5 соединена только с v_4 и является концевой, доминирующих вершин нет, так как ни одна вершина не имеет степени 5. Сумма степеней равна 12, что вдвое больше размера графа (6), как и должно быть.



```
Степени вершин:
deg(v1) = 2
deg(v2) = 3
deg(v3) = 3
deg(v4) = 3
deg(v5) = 1
deg(v6) = 0

Изолированные вершины: [6]
Концевые вершины:      [5]
Доминирующие вершины:  нет
```

Рисунок 6 — Степени и классификация вершин по матрице смежности

Результат выполнения задания 2 для того же графа: построена матрица инцидентности размера 6×6 и список рёбер, размер графа определён по числу столбцов.

Матрица инцидентности:						
	e1	e2	e3	e4	e5	e6
v1	1	1	0	0	0	0
v2	1	0	1	1	0	0
v3	0	1	1	0	1	0
v4	0	0	0	1	1	1
v5	0	0	0	0	0	1
v6	0	0	0	0	0	0

Список рёбер:						
e1	=	(v1,	v2)			
e2	=	(v1,	v3)			
e3	=	(v2,	v3)			
e4	=	(v2,	v4)			
e5	=	(v3,	v4)			
e6	=	(v4,	v5)			

Размер графа	=	6
--------------	---	---

Рисунок 7 — Матрица инцидентности, список рёбер и размер графа (задание 2)

Степени вершин и классификация вершин по матрице инцидентности полностью совпали с результатами, полученными по матрице смежности.

Степени вершин:	
deg(v1)	= 2
deg(v2)	= 3
deg(v3)	= 3
deg(v4)	= 3
deg(v5)	= 1
deg(v6)	= 0

Изолированные вершины:	[6]
Концевые вершины:	[5]
Доминирующие вершины:	нет

Рисунок 8 — Степени и классификация вершин по матрице инцидентности

Так как доминирующая вершина и изолированная вершина не могут существовать в одном графе одновременно, для проверки определения доминирующих вершин программа была запущена ещё раз на другом графе. В нём вершина v1 смежна со всеми остальными вершинами (степень 5) и определена как доминирующая, а вершина v6 имеет степень 1 и определена как концевая.

```

Матрица смежности графа G:
      v1 v2 v3 v4 v5 v6
v1    0  1  1  1  1  1
v2    1  0  1  0  0  0
v3    1  1  0  0  0  0
v4    1  0  0  0  1  0
v5    1  0  0  1  0  0
v6    1  0  0  0  0  0

Размер графа = 7

Степени вершин:
deg(v1) = 5
deg(v2) = 2
deg(v3) = 2
deg(v4) = 2
deg(v5) = 2
deg(v6) = 1

Изолированные вершины:  нет
Концевые вершины:      [6]
Доминирующие вершины:  [1]

```

Рисунок 9 — Результат для графа с доминирующей и концевой вершинами

Во всех случаях результаты, выданные программой, совпали с ожидаемыми, полученными вручную по рисунку графа.

Вывод:

В ходе выполнения лабораторной работы были изучены основные характеристики графов и способы их представления. На языке Go реализованы генерация матрицы смежности неориентированного графа, построение матрицы инцидентности, определение размера графа, вычисление степеней вершин и поиск изолированных, концевых и доминирующих вершин. Показано, что размер графа по матрице смежности равен половине суммы всех её элементов, а по матрице инцидентности — числу столбцов; степени вершин в обоих случаях равны суммам элементов строк. Результаты, полученные по двум представлениям графа, совпали, что подтверждает корректность реализации.